Национална програма
"Обучение за ИТ умения и кариера"
https://it-kariera.mon.bg

Министерството на
образованието и науката
https://www.mon.bg

Хвърляне на изключения

Exceptions

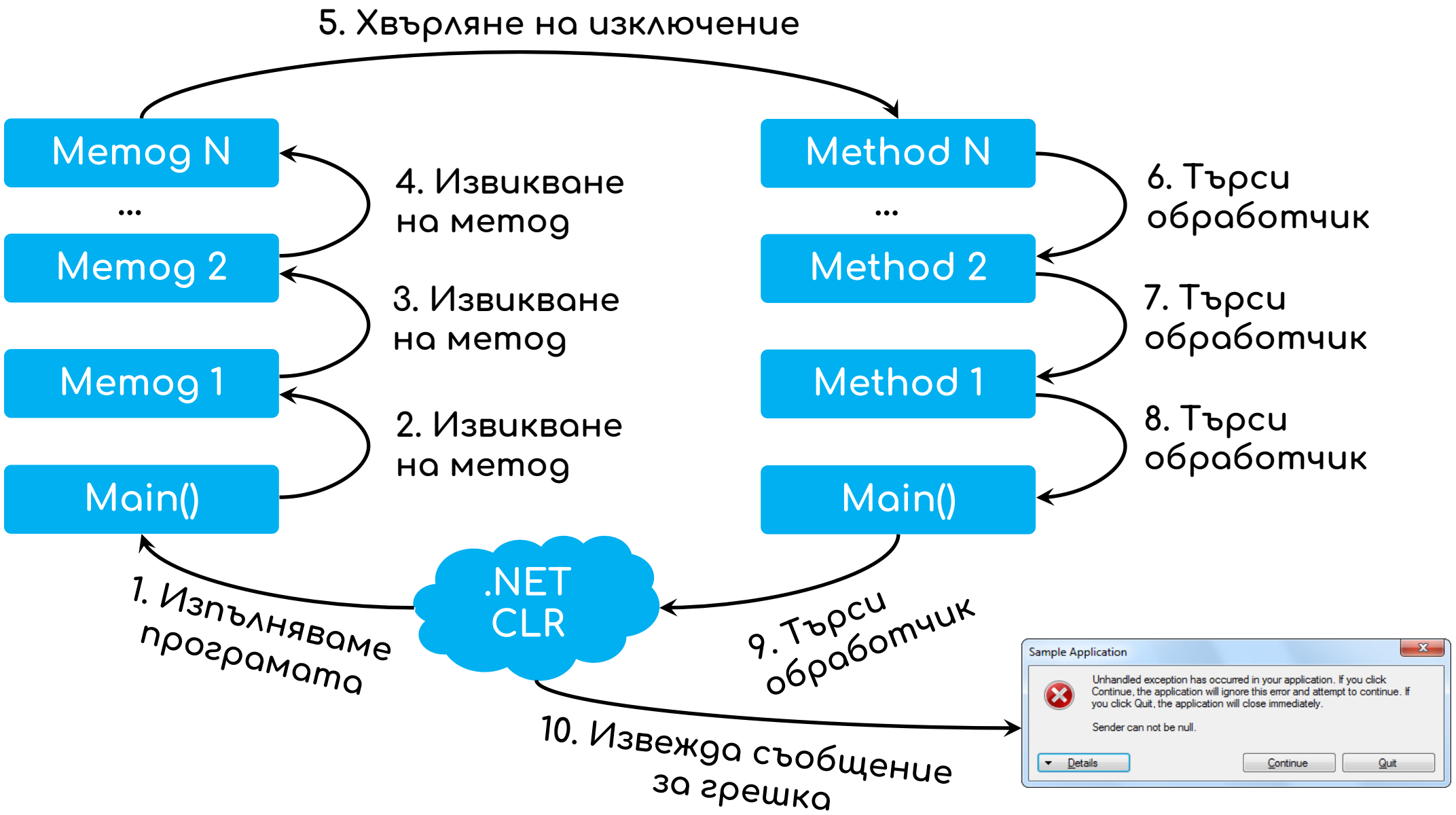
Съдържание

1. Генериране (хвърляне) на изключения
2. Избор на типа на изключението
3. Препоръки при работа с изключения
4. Създаване на потребителски изключения

Хвърляне на изключения

- Изключенията се хвърлят (пораждат) чрез командата throw
- Целта е уведомяване на кода, извикал текущия програмен блок, за грешка или друга необичайна ситуация
- Когато се хвърля изключение:
  - Изпълнението на програмата спира
  - Изключението пътува през стека
    - Докато не достигне подходящ catch блок, който да го прихване
- Неприхванатите изключения извеждат съобщение за грешка

Как работят изключенията?



Използване на командата Throw

- Хвърляне на изключение със съобщение за грешка:

```
throw new ArgumentException("Invalid amount!");
```

- Изключението може да приема съобщение и причина:

```
try
{
  …
}
catch (SqlException sqlEx)
{
  throw new InvalidOperationException("Cannot save invoice.", sqlEx);
}
```

- Бележка: ако и оригиналното изключение не бъде подадено като параметър, ще загубим първоначалната причина за изключението

Повторно хвърляне на изключение

Прихванатите изключения може да бъдат хвърлени наново:

```
try
{
  Int32.Parse(str);
}
catch (FormatException fe)
{
  Console.WriteLine("Parse failed!");
  throw fe; // Re-throw the caught exception
}
```

```
catch (FormatException)
{
  throw; // Re-throws the last caught exception
}
```

Хвърляне на изключения – пример

```
public static double Sqrt(double value)
{
   if (value < 0)
       throw new System.ArgumentOutOfRangeException("value",
          "Sqrt for negative numbers is undefined!");
   return Math.Sqrt(value);
}

static void Main()
{
   try
   {
       Sqrt(-1);
   }
   catch (ArgumentOutOfRangeException ex)
   {
       Console.Error.WriteLine("Error: " + ex.Message);
       throw;
   }
}
```

Избиране на типа на изключението

- Когато се подаде невалидна стойност в параметър на метод:
  - ArgumentException, ArgumentNullException, ArgumentOutOfRangeException
- Когато заявената операция не се поддържа
  - NotSupportedException
- Когато методът все още не е реализиран
  - NotImplementedException
- Когато няма друг подходящ стандартен клас изключения
  - Създайте ваш собствен клас (наследяващ Exception)

Препоръки при работа с изключения [1/3]

- catch блоковете трябва да започват с изключенията, които са най-ниско в йерархията (т.е. с най-специфичните)
  - И да продължават с по-общите изключения
  - В противен случай ще има грешка при компилация
- Всеки catch трябва да обработва само тези изключения, които очаква
  - Ако метод не е компетентен да обработи дадено изключение, той би трябвало да го остави неприхванато
  - Прихващането на всички изключения, независимо от какъв тип са, е популярна лоша практика (анти-шаблон)!

Препоръки при работа с изключения [2/3]

- Когато генерирате изключение, винаги подавайте на конструктора достатъчно говорящо **пояснително съобщение**
- Когато хвърляте изключение, винаги подавайте добро описание на проблема, който го е предизвикал
  - Съобщението на изключението трябва да обяснява какво е породило проблема и как той може да бъде решен
  - Добро: „*Размерът трябва да е число в диапазона [1…15]*"
  - Добро: „*Невалидно състояние. Извикайте първо Initialize()*"
  - Лошо: „*Неочакван проблем*"
  - Лошо: „*Невалиден аргумент*"

Препоръки при работа с изключения [3/3]

- Изключенията може да намалят производителността на приложението
  - Затова ги хвърляйте само в ситуации, които са наистина необичайни и трябва да бъдат обработени
  - Не хвърляйте изключения при нормалната работа на програма
  - CLR може да хвърли изключения по всяко време, няма как да бъде предвидено това
    - Например System.OutOfMemoryException

Създаване на потребителски изключения

- Потребителските изключения наследяват някой от класовете изключения (най-често System.Exception)

```
public class TankException : Exception
{
    public TankException(string msg)
        : base(msg)
    {
    }
}
```

- Те се хвърлят като всяко друго изключение

```
throw new TankException("Not enough fuel to travel");
```

Национална програма
"Обучение за ИТ умения и кариера"
https://it-kariera.mon.bg

Министерството на
образованието и науката
https://www.mon.bg

Документът е разработен за нуждите на Национална програма "Обучение за ИТ умения и кариера" на Министерството на образованието и науката (МОН) и се разпространява под свободен лиценз CC-BY-NC-SA (Creative Commons Attribution-Non-Commercial-Share-Alike 4.0 International).